# Grokking, visualized: an explorable, reproducible account of the modular-addition transformer, with a preregistered multi-seed replication

Libor Burian
burnycoder.github.io/visualizing-grokking-modular-addition · code

**Abstract.** Nanda et al. [1] reverse-engineered a one-layer transformer that groks addition modulo 113 and showed that it computes $(a + b) \bmod 113$ with a Fourier-multiplication algorithm. We (i) re-implement their analysis and that of the ARENA course [2], and check it against 30 numbers they publish for their own network: 26 reproduce, and we explain the 4 that do not; (ii) add a counterfactual causal test — swapping the key-frequency subspace of the embedding of $a$ with that of $a'$ moves the output to $a' + b$, recovering 0.997 of the logit difference (95 % CI 0.994–1.000) versus 0.002 for the complementary subspace and 0.062 for a random subspace of equal dimension; (iii) retrain the paper's exact recipe for 11 seeds plus no-weight-decay, random-label and fixed-split controls, deterministically and bit-reproducibly, and test five preregistered hypotheses: H1 supported (11/11 seeds; Holm-adjusted p = 0.002); H2 supported (11/11 seeds; Holm-adjusted p = 0.002); H3 supported (11/11 seeds; Holm-adjusted p = 0.002); H4 not supported (6/11 seeds; Holm-adjusted p = 0.5); H5 supported (11/11 weight-decay runs vs 0/5 without grokked; Holm-adjusted p = 0.0011). (iv) publish an explorable website in which every step of the evidence is shown on the real weights, every term is defined before use, and every number is bound to the analysis output.

## 1 Introduction

**Grokking** is delayed generalisation: a network first fits its training data while test performance stays at chance, and much later, without any change to the training setup, suddenly generalises [3]. Nanda et al. [1] studied the smallest well-understood instance — a one-layer transformer trained on $a + b \bmod 113$ — and showed that the trained network implements a specific, human-readable algorithm, and that measures derived from this algorithm ("progress measures") move smoothly long before test loss falls.

Their evidence comes from one training run (the "mainline"), with appendix checks on four further seeds. Two things are missing for a reader who wants to verify the story: (a) a way to *see* every intermediate object on the real network, and (b) a pre-specified, statistically controlled test of the main claims across independently trained networks. This work provides both.

**Contributions.**

1. An analysis pipeline whose outputs match 26 of 30 published numbers on the paper's own checkpoints (Section 4.1), with documented definitional findings (e.g., the reported "test loss" $2.41 \times 10^{-7}$ is the loss over all 12,769 pairs).
2. A counterfactual subspace-patching test of the claim that the embedding encodes $a$ only through $\cos(w_k a), \sin(w_k a)$ at the key frequencies (Section 4.3).
3. A preregistered replication over 11 seeds with controls, trained deterministically (Section 4.5).
4. An explorable website with 22 chapters and a dashboard of linked figures, built so that the text cannot drift from the data (Section 3.4).

## 2 Background

**Task.** All $113^2$ = 12,769 pairs $(a, b)$ are written as tokens `a b =`; the label is $c^* = (a + b) \bmod 113$. A fixed random 3,830 pairs (30 %) are used for training, the remaining 8,939 for testing.

**Model** [1, Sec. 3]. One transformer layer with $d_{\text{model}} = 128$, four attention heads of size 32, an MLP with 512 ReLU neurons, no LayerNorm, untied embedding $W_E$ and unembedding $W_U$; 226,816 parameters. It is trained full-batch with AdamW [4] (learning rate $10^{-3}$, weight decay $\lambda = 1$, $\beta = (0.9, 0.98)$) for 40,000 steps.

**The algorithm** [1, Sec. 4]. For a few **key frequencies** $k$ (with $w_k = 2\pi k/113$), the embedding maps $a$ to $\cos(w_k a), \sin(w_k a)$; attention and MLP compute $\cos(w_k(a + b))$ and $\sin(w_k(a + b))$ using $\cos(x + y) = \cos x \cos y - \sin x \sin y$; the unembedding reads out

$$\text{logit}(c) \approx \sum_{k \in K} \alpha_k \cos(w_k(a + b - c)),$$

which is maximal exactly at $c = c^*$. All frequencies' cosines peak together there and interfere destructively elsewhere.

**Progress measures** [1, Sec. 5]. The **restricted loss** keeps only the key-frequency components of the logits (it measures how well the algorithm alone would do); the **excluded loss** removes them and evaluates on training data (it measures how much the network relies on memorisation instead). Training splits into **memorisation**, **circuit formation** and **cleanup** phases by these measures.

# 3 Methods

## 3.1 The paper's run and our analysis code

We use the authors' published mainline run [5] (500 checkpoints, pinned revision and SHA-256). Our model re-implements the paper's parameter layout so the checkpoints load unmodified. Recomputing the training loss from the checkpoints shows that the checkpoint labelled "epoch $e$" holds the parameters after $e + 1$ optimiser updates (the learning-rate warm-up starts at zero) and confirms that we rebuilt the exact training split. Analysis functions are re-implemented from the paper's code [6] and ARENA [2], each with a source link; exact definitions are in `docs/operationalizations.md` of the repository.

## 3.2 Counterfactual subspace patching

Let $Q \in \mathbb{R}^{128 \times 2|K|}$ be an orthonormal basis of the embedding directions $W_E F_{2k-1}, W_E F_{2k}$ ($F$: the Fourier basis over $\mathbb{Z}_{113}$). For a triple $(a, a', b)$ we replace the component of $a$'s embedding in span($Q$) by that of $a'$ and measure the recovered fraction of the logit difference $D = L[a' + b] - L[a + b]$: $\big(D_{\text{patched}} - D(a)\big)/(D(a') - D(a))$ $-$ 1 means the network now answers $a' + b$. Controls: the complementary subspace, and a random $2|K|$-dimensional subspace. 1,000 random triples; bootstrap 95 % intervals. This is an interchange intervention in the sense of [7].

## 3.3 Training and preregistration

We trained 22 runs with the paper's recipe: `wd1` (11 seeds; seed 0 uses the paper's split), `wd0` (5 seeds, $\lambda = 0$), `rand` (3 seeds, random label table) and `split0` (3 new initialisations on the paper's split). All runs train together with vectorised forward/backward passes; batched and single-run updates agree, runs are bitwise independent of their batch partners, and training is bitwise reproducible across processes and resumption. Early full-batch Adam training amplifies rounding differences (a mathematically identical forward pass with a different summation order diverges from $3 \times 10^{-7}$ after 20 steps to 0.24 after 200 steps in max parameter difference), so seed-level results must be compared as distributions, not trajectories.

Before training, we fixed five hypotheses, their metrics and falsification rules (git tag `prereg-v1`): **H1** restricted loss falls before test loss; **H2** excluded loss rises ≥ 10× during circuit formation; **H3** the final Gini sparsity [8] of the $W_E$ and $W_L$ Fourier spectra exceeds both its initial value + 0.3 and the 99th percentile under 100 random orthonormal bases; **H4** each single key-frequency ablation raises loss ≥ 100×, each non-key ablation ≤ 2×, removing all non-key frequencies ≤ 1.5×; **H5** weight decay is necessary (all `wd1` and no `wd0` runs reach 99 % test accuracy). H1–H4 use one-sided exact sign tests over the 11 `wd1` seeds, H5 a one-sided Fisher exact test; the five p-values are Holm-corrected [9] at $\alpha = 0.05$. Eleven seeds is the smallest number at which one dissenting seed still passes the strictest Holm threshold ($P(X \geq 10 \mid 11, 1/2) = 0.0059 < 0.01$).

## 3.4 Website

The site is generated from Markdown with macros: `[[dfn:…]]` defines a term, `[[term:…]]` uses one, and the build fails if any term or symbol is used before its definition in reading order, if a quote does not match its PDF page, or if a number is not bound to `summary.json`. Every figure is one JavaScript module shared by the chapters and the dashboard; per-input views run a JavaScript copy of the model on float16 weights, tested against PyTorch. The figures in this paper are screenshots of the same modules.

# 4 Results

## 4.1 Reproducing the published numbers

Table 1 lists all 30 checks. 26 match within stated tolerances. Two definitional findings: the paper's "test loss" $2.41 \times 10^{-7}$ equals the loss over *all* pairs (test-only: $3.02 \times 10^{-7}$), and the "$W_L$ residual < 0.55 %" is a squared-norm ratio (0.6

%; the norm ratio is 7.4 %). The four non-reproduced rows are projections and ablations whose exact procedure the paper does not specify; for R10b, the paper's value equals our restricted-*logit* loss rather than a per-neuron restriction.

| ID | Quantity | Paper / ARENA | Ours | Match |
|---|---|---|---|---|
| R1 | loss at epoch 40k (all 12,769 pairs, 113-way) | 2.41e-07 | 2.412e-07 | yes |
| R2 | key frequencies | [14, 35, 41, 42, 52] | [14, 35, 41, 42, 52] | yes |
| R3 | non-negligible W_E frequencies | 6 | 6 | yes |
| R4 | W_L residual after 10-direction approximation | <0.55 % | 0.005517 | yes |
| R5 | neurons with >85 % FVE at one frequency | 433 | 433 | yes |
| R6 | neurons in the k=14 cluster | 44 | 44 | yes |
| R7 | Table 1 FVE of u_k/v_k readoffs (mean of 10) | 0.9624 | 0.9635 | yes |
| R8a $\Sigma\alpha_k \cos(w_k(a+b-c))$ FVE | logit fit | 0.95 | 0.9614 | yes |
| R8b | loss of the fitted logits | 4.7e-08 | 4.395e-08 | yes |
| R9a | loss keeping only the 40 key 2D components | 7.24e-08 | 7.236e-08 | yes |
| R9b | MLP projected onto 10 W_L directions | 1.19e-07 | 2.377e-07 | no |
| R9c | MLP projected onto the null- | 5.27 | 4.804 | no |
| [illegible] | [illegible] | [0.000282, 6[illegible]95, 0.03288] | [[0.004418, 0.001[illegible]9146, 0.0008718]] | yes |
| sizes 1[illegible]42/52/ to co[illegible](a+b)) | | | | |

Table 1. Reproduction checks on the paper's network at epoch 40k (step 40,001). Sources: paper pages and ARENA tests as listed in `reports/tables/exp01_rchecks.md`.

## 4.2 The mechanism on the paper's network

The embedding is sparse in the Fourier basis: six frequencies carry visible weight (14, 31, 35, 41, 42, 52), five of which recur everywhere in the network and are the key frequencies 14, 35, 41, 42, 52 (Figure 1). 433 of 512 MLP neurons have more than 85 % of their variance explained by a single frequency, forming clusters of k = 14: 44, k = 35: 93, k = 41: 145, k = 42: 87, k = 52: 64 neurons. The logits are fit by $\sum_k \alpha_k \cos\left(w_{k(a+b-c)}\right)$ with 96.1 % variance explained; replacing the logits by this fit gives loss $4.40\times10^{-8}$. Ablating any single key frequency from the logits multiplies the loss by 4,315× to 315,210×; the largest effect of a non-key frequency is 1.09× (Figure 2).

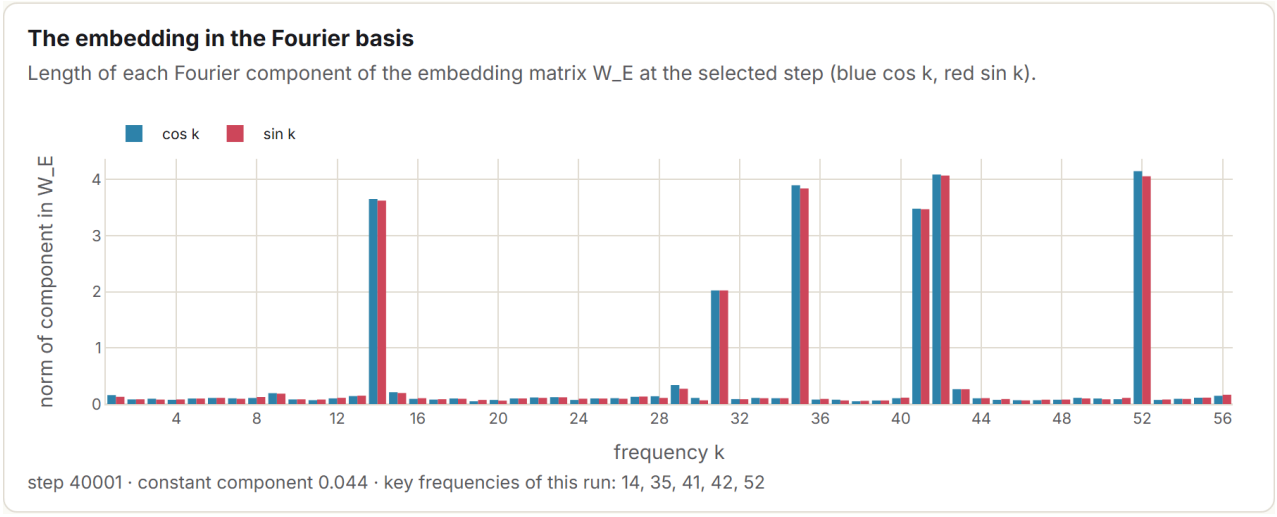



Figure 1: Norm of each Fourier component of the embedding $W_E$ at step 40,001 (paper run).

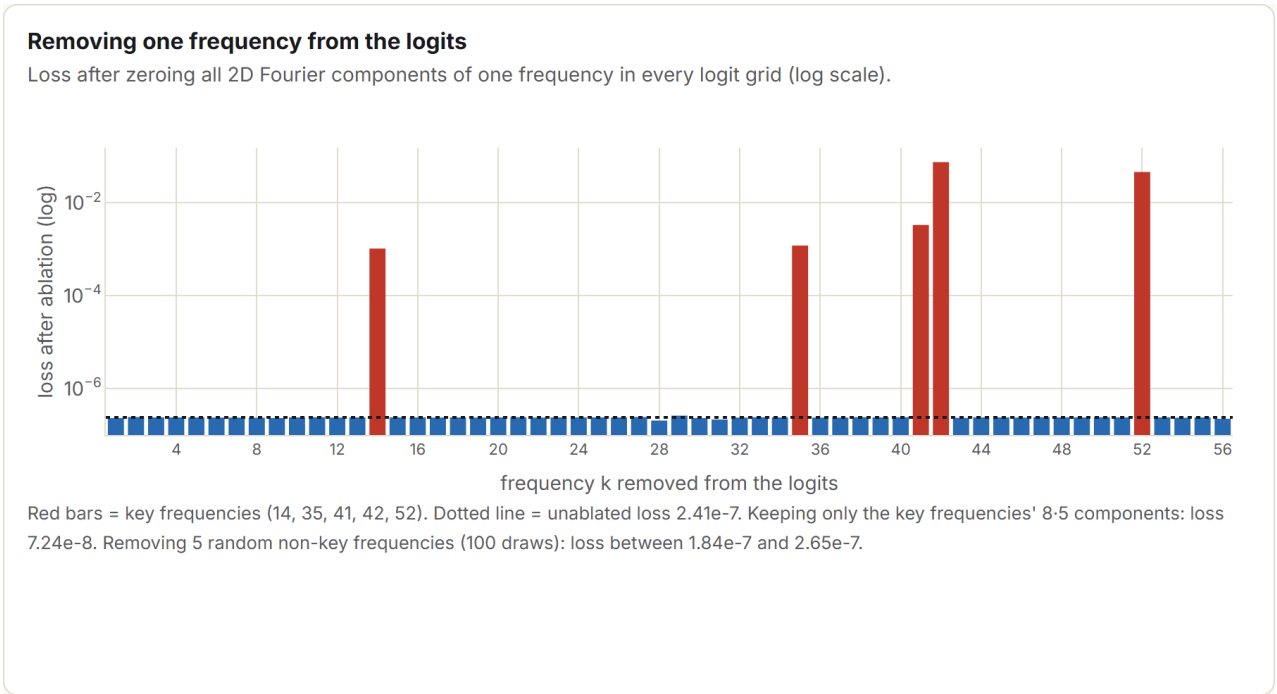



Figure 2: Loss after removing all components of one frequency from the logits (log scale), paper run at step 40,001. Red bars: the key frequencies; dotted line: the unablated loss.

## 4.3 The causal test

Swapping the key-frequency subspace (dimension 10 of 128) recovers 0.997 of the logit difference (95 % CI 0.994–1.000); swapping everything *except* that subspace recovers 0.002, and a random 10-dimensional subspace 0.062 — close to the $10/128 \approx 0.078$ expected if the random subspace captured a proportional share of the relevant direction (Figure 3). The number $a$ therefore enters the computation essentially only through its key-frequency components.

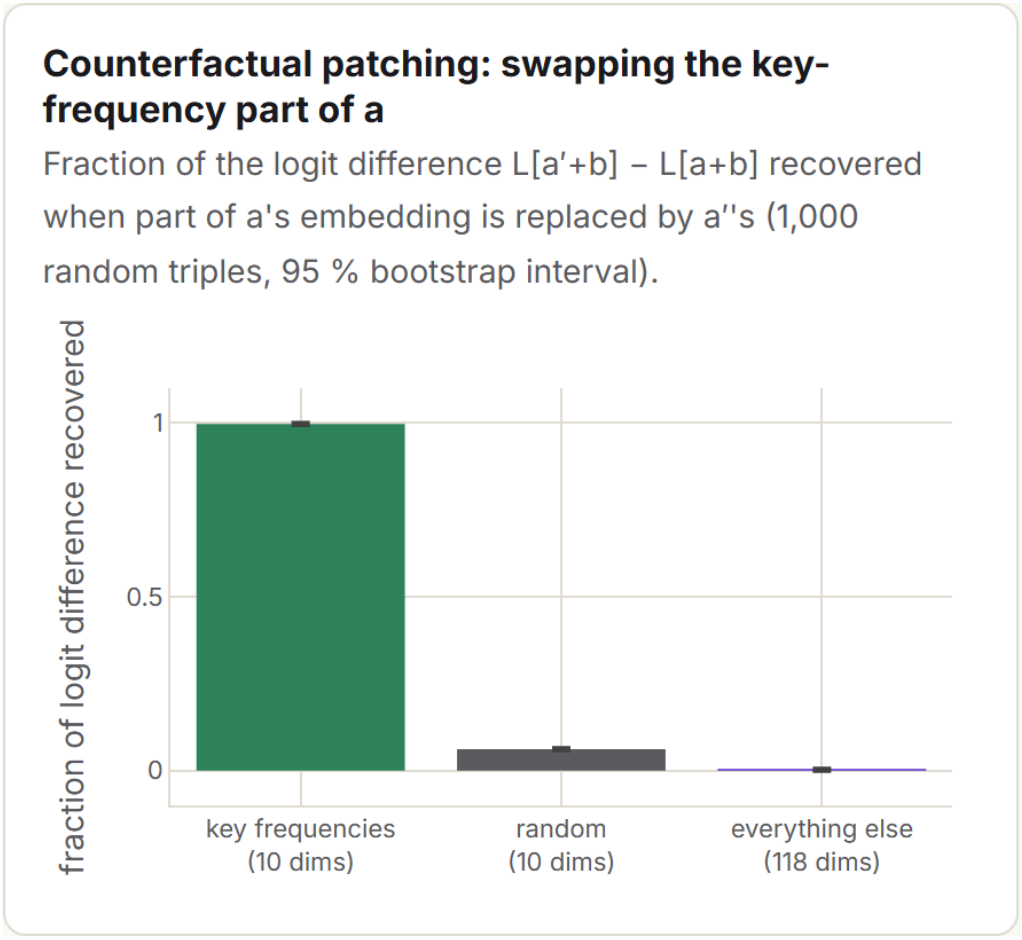



Figure 3: Counterfactual subspace patching on the paper run: recovered fraction of the logit difference for the key-frequency subspace and the two controls (1,000 random triples; bootstrap 95 % intervals).

### 4.4 Training dynamics

On the paper's run our objective phase rules put the end of memorisation at step 1,567, the end of circuit formation at 9,301, and the end of cleanup at 14,001; test accuracy reaches 99 % at 10,601 (Figure 5). The excluded loss rises from $6.94\times10^{-5}$ at the end of memorisation to 8.10 at the end of circuit formation, while the restricted loss has already fallen far below the test loss. Exploratory gradient analysis (exact full-batch gradients at every checkpoint): the cosine between the training-set and test-set gradients is 0.80 at initialisation and becomes negative during memorisation and circuit formation (-0.27 at step 5,401); after grokking the gradient is almost exactly anti-parallel to the weights (cosine -0.996 at step 40,001), the signature of an equilibrium between the loss gradient and weight decay.

Across our 11 seeds (exploratory; reports/exp-04) the train-test gradient cosine is negative at 97.9 % [96.0 %, 99.6 %] of the checkpoints between $t_1$ and $t_2$. Because Adam rescales gradients, we also rebuilt the optimiser's actual direction $d = \hat{m}/\left(\sqrt{\hat{v}} + \varepsilon\right)$ from moments saved every 1,000 steps. At step 40,000 the loss step and the decay step are nearly equal and opposite: $\|d\|/(\lambda\|\theta\|)$ = 0.992 [0.972, 1.004], $\cos(d, \theta)$ = -0.991 [-1.000, -0.976], whereas the raw gradient's cosine with the weights varies from -0.996 to -0.157. The ratio is lowest, 0.906 [0.897, 0.914], within 1,000 steps of grokking in 10 of 11 seeds: weight decay briefly outweighs the loss while the network generalises, consistent with the paper's account of cleanup. This last observation is post hoc.




Figure 4: Size of Adam's loss step relative to the weight-decay step for seed 0, every 1,000 steps (log scale); shading marks the phases.

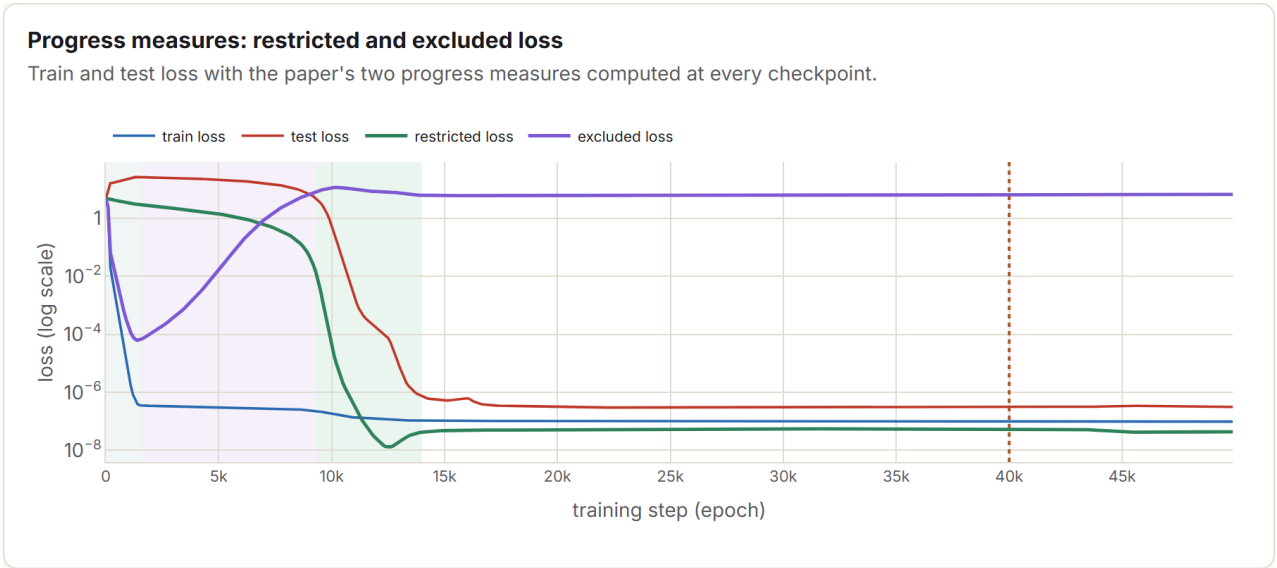



Figure 5: Train, test, restricted and excluded loss over training of the paper run (log scale); shading marks the three phases from our objective rules.

## 4.5 Replication across seeds and controls

| H | Result | Holm p | Supported |
|---|---|---|---|
| H1 | 11 / 11 seeds | 0.002 | yes |
| H2 | 11 / 11 seeds | 0.002 | yes |
| H3 | 11 / 11 seeds | 0.002 | yes |
| H4 | 6 / 11 seeds | 0.5 | no |
| H5 | 11/11 with vs 0/5 without weight decay grokked | 0.0011 | yes |

Table 2: Preregistered hypotheses over the 11 `wd1` seeds (H5: `wd1` vs `wd0`).

Across the 11 weight-decay seeds, memorisation ends at step 1,576 [1,567, 1,586], circuit formation at 7,445 [5,874, 9,017], and test accuracy reaches 99 % at 8,455 [6,816, 10,093] (mean, 95 % t-interval). Each run uses 4.364 [3.743, 4.985] key frequencies; single-frequency neurons make up 80.2 % [75.3 %, 85.2 %] of the MLP, the cosine fit explains 96.5 % [95.3 %, 97.7 %] of the logit variance, and the patching test recovers 0.997 [0.995, 0.998] for the key subspace versus 0.058 [0.049, 0.067] for a random one.

**Why H4 fails.** H4 conjoins three conditions. Decomposing the preregistered criterion post hoc (counts only, no new test): every non-key frequency is individually dispensable in 11/11 seeds (worst 1.84×); every key frequency is necessary in 9/11 (weakest 5.9×); keeping only the key frequencies costs ≤ 1.5× in 8/11 (worst 2.78×). Both failing parts concern the operationalisation of $K$ — frequencies whose $W_L$ Fourier norm is ≥ 25 % of the largest — rather than the existence of a sparse set of necessary frequencies: the rule twice admits a frequency with a large weight norm but small causal effect, and in seeds with 3–4 key frequencies the many individually negligible non-key components jointly matter at the $10^{-7}$ loss scale. Thresholds calibrated on a single model with five well-separated frequencies did not transfer to all seeds; a causal (ablation-based) definition of $K$ would be the natural preregistered choice in future work.

**Controls.** No weight decay: final train accuracy 100 %, 100 %, 100 %, 100 %, 100 %, final test accuracy 6.0 %, 1.9 %, 8.1 %, 21.5 %, 16.2 %; Gini of the embedding spectrum 0.07, 0.07, 0.08, 0.07, 0.07 (initially ≈ 0.03). Random labels: final train accuracy 100 %, 100 %, 100 %, final test accuracy 0.9 %, 0.9 %, 0.8 %; Gini of the embedding spectrum 0.09, 0.09, 0.08 (initially ≈ 0.04). Paper split with new initialisations: final test accuracy 100.0 %, 100.0 %, 100.0 %; key frequencies split0_init100: [2, 14, 27, 30, 37, 52]; split0_init101: [9, 14, 15, 27]; split0_init102: [7, 38, 49, 53] (the paper's run: {14, 35, 41, 42, 52}).

**Exploratory: what selects the frequencies.** Our seed 0 starts from the paper's exact initial weights (max difference $2.98\times10^{-8}$) on the paper's split. The two training-loss curves agree to five significant digits for 52 steps and then diverge — full-batch Adam amplifies rounding differences — and the final weights have cosine similarity 0.74; nevertheless both runs end with the same key frequencies (14, 35, 41, 42, 52), whereas three new initialisations on the same split chose different ones. With a single same-initialisation pair this is suggestive only: the initialisation, not floating-point noise, appears to select the frequencies.

**Exploratory: late loss spikes.** After grokking, the test loss of 2 of 14 grokked runs repeatedly jumps more than 100× above its running minimum and recovers: split0_init102 (6 spikes; test loss up to 0.64, test accuracy down to 87 %); wd1_s4 (4 spikes; test loss up to 0.25, test accuracy down to 93 %). Such spikes with adaptive optimisers were described as the slingshot mechanism [10]; we did not investigate their cause. All tests use step 40,000, where both runs are at 100 % test accuracy.

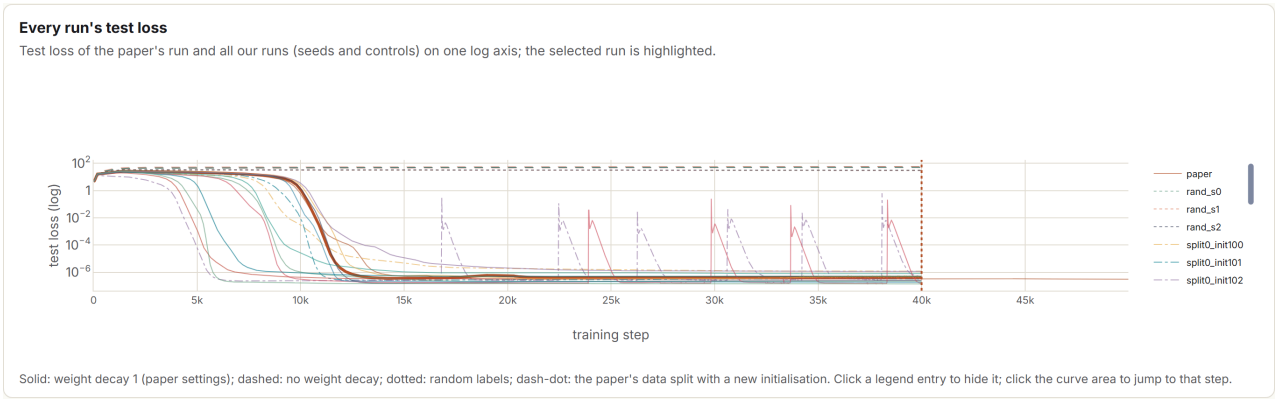



Figure 6: Test loss of every run over training (log scale); seeds as thin lines.

# 5 Related work

Power et al. [3] introduced grokking on algorithmic tasks. Liu et al. [11] explain it through representation learning dynamics; Thilak et al. [10] link it to adaptive-optimiser instabilities; Barak et al. [12] show "hidden progress" in parity learning. Nanda et al. [1] give the mechanistic account we test; Chughtai et al. [13] generalise it to group operations, Zhong et al. [14] show that architecturally similar networks may instead learn a different ("pizza") algorithm, and Gromov [15] gives an analytic construction for MLPs. Interactive explanations exist as the PAIR explorable [16] and a Welch Labs video [17]; ARENA [2] provides exercises. Our contribution is a verified, preregistered replication of the mechanistic claims together with an explorable presentation of the full evidence.

# 6 Discussion and limitations

**What the evidence supports.** On the paper's network, the reproduction and the patching test agree with the Fourier-multiplication account to within a few percent; the patching result is the most direct causal statement we make: $a$ is read through a 10-dimensional subspace of a 128-dimensional embedding. Across seeds, hidden progress (H1), the dismantling of memorisation (H2), learned basis-specific sparsity (H3) and the necessity of weight decay (H5) replicate in every seed, and the patching result replicates with the same effect size. The one failed test (H4) shows that "the key frequencies" is less sharply defined than the paper's single model suggests: which frequencies count depends on whether one ranks them by weight norm or by causal effect.

**Limitations.** (i) All runs share the paper's architecture, data fraction, modulus and optimiser; we do not test other settings, where other algorithms are known to arise [14]. (ii) Four of the paper's numbers could not be reproduced because procedures are under-specified; we do not use them as evidence. (iii) Phase boundaries come from rules we calibrated on the paper's run before seeing our runs; other reasonable rules shift them. (iv) Our runs last 40,000 steps; the paper's late change near step 43,000 is outside this window. (v) Training is bitwise reproducible only on the same software and hardware stack. (vi) Our seed 0 shares the paper's initialisation and split, so it is not independent of the paper's run (it is independent of our other seeds). (vii) The H4 decomposition, the same-initialisation comparison and the loss spikes are post hoc and descriptive.

# 7 Conclusion

On the paper's network, the Fourier-multiplication account survives a counterfactual test: the input is read through its key-frequency subspace and essentially nowhere else. Across 11 independently trained seeds, four of five preregistered claims replicate; the fifth fails on how sharply "key frequencies" can be delimited, not on whether a few frequencies carry the computation. The website makes every step of this evidence inspectable on the real weights; the pipeline, data and preregistration make it reproducible.

# Bibliography

[1] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Progress measures for grokking via mechanistic interpretability," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2301.05217

[2] ARENA, "Chapter 1.5.2: Grokking and Modular Arithmetic." [Online]. Available: https://learn.arena.education/chapter1_transformer_interp/32_grokking/intro

[3] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets." [Online]. Available: https://arxiv.org/abs/2201.02177

[4] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[5] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Mainline training run of ``Progress measures for grokking'' (checkpoints and losses)." [Online]. Available: https://huggingface.co/callummcdougall/grokking_full_run_data

[6] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "progress-measures-paper (code)." [Online]. Available: https://github.com/mechanistic-interpretability-grokking/progress-measures-paper

[7] A. Geiger, H. Lu, T. Icard, and C. Potts, "Causal Abstractions of Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. [Online]. Available: https://arxiv.org/abs/2106.02997

[8] N. Hurley and S. Rickard, "Comparing Measures of Sparsity," *IEEE Transactions on Information Theory*, vol. 55, no. 10, pp. 4723–4741, 2009, [Online]. Available: https://arxiv.org/abs/0811.4706

[9] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure." [Online]. Available: https://www.jstor.org/stable/4615733

[10] V. Thilak, E. Littwin, S. Zhai, O. Saremi, R. Paiss, and J. Susskind, "The Slingshot Mechanism: An Empirical Study of Adaptive Optimizers and the Grokking Phenomenon." [Online]. Available: https://arxiv.org/abs/2206.04817

[11] Z. Liu, O. Kitouni, N. Nolte, E. J. Michaud, M. Tegmark, and M. Williams, "Towards Understanding Grokking: An Effective Theory of Representation Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2205.10343

[12] B. Barak, B. L. Edelman, S. Goel, S. Kakade, E. Malach, and C. Zhang, "Hidden Progress in Deep Learning: SGD Learns Parities Near the Computational Limit," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://arxiv.org/abs/2207.08799

[13] B. Chughtai, L. Chan, and N. Nanda, "A Toy Model of Universality: Reverse Engineering How Networks Learn Group Operations." [Online]. Available: https://arxiv.org/abs/2302.03025

[14] Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas, "The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2306.17844

[15] A. Gromov, "Grokking modular arithmetic." [Online]. Available: https://arxiv.org/abs/2301.02679

[16] A. Pearce, A. Ghandeharioun, N. Hussein, N. Thain, M. Wattenberg, and L. Dixon, "Do Machine Learning Models Memorize or Generalize?." [Online]. Available: https://pair.withgoogle.com/explorables/grokking/

[17] Welch Labs, "The most complex model we actually understand." [Online]. Available: https://www.youtube.com/watch?v=D8GOeCFFby4

## Appendix A: Reproducibility

Code, data formats and the full pipeline: `github.com/BurnyCoder/visualizing-grokking-modular-addition` (`uv run grokviz --phases all`). Runs: Hugging Face `BurnyCoder/grokking-modular-addition-runs`; site data: `BurnyCoder/grokking-modular-addition-site-data`. Hardware: one RTX 5070 Laptop GPU; training of all 22 runs takes ≈ 73 minutes, per-checkpoint analysis ≈ 72 minutes. Per-experiment reports with learnings are in `reports/`.